

WYDZIAŁ
ELEKTROTECHNIKI
I INFORMATYKI
POLITECHNIKI RZESZOWSKIEJ

Kacper Hyliński

Implementacja algorytmu DQN do budowania strategii agenta gry komputerowej

PRACA INŻYNIERSKA

Opiekun pracy:
dr hab. inż. Roman Zajdel prof. PRz

Rzeszów, 2026

Spis treści

Wykaz symboli, oznaczeń i skrótów **5**

1. Wstęp **7**

1.1. Motywacja i zarys problemu 7

1.2. Struktura pracy 8

1.3. Cel i zakres pracy 8

2. Uczenie ze wzmocnieniem w grach komputerowych **9**

2.1. Podstawy uczenia ze wzmocnieniem 9

2.1.1. Proces decyzyjny Markowa 9

2.1.2. Zwrot i współczynnik dyskontowania 10

2.1.3. Strategia i funkcje wartości 10

2.1.4. Równania Bellmana 10

2.2. Sieci neuronowe 11

2.2.1. Model neuronu 11

2.3. Gry jako środowisko badawcze 12

2.4. Przykłady zastosowań 12

3. Przegląd algorytmów i bibliotek **13**

3.1. Algorytmy tablicowe 13

3.2. Głębokie algorytmy oparte na wartości 13

3.3. Metody gradientu strategii 13

3.4. Biblioteki uczenia ze wzmocnieniem 13

3.5. Uzasadnienie wyboru rozwiązania 13

4. Projekt systemu **15**

4.1. Środowisko gry Snake 15

4.2. Reprezentacja stanu 15

4.3. Funkcja nagrody 15

4.4. Architektura systemu 15

5. Implementacja **17**

5.1. Implementacja środowiska gry 17

5.2. Sieć neuronowa 17

5.3. Agent DQN 17

5.4. Proces uczenia 17

5.5. Interfejs graficzny 17
6. Podsumowanie i wnioski 19
Literatura 20

Wykaz symboli, oznaczeń i skrótów

1. Wstęp

1.1. Motywacja i zarys problemu

Gry komputerowe od lat stanowią jedno z najważniejszych środowisk badawczych sztucznej inteligencji. Mają jasno określone reguły, mierzalny wynik i pozwalają przeprowadzić tysiące rozgrywek w czasie, w którym człowiek rozegrałby zaledwie kilka. Te cechy sprawiają, że gry dobrze nadają się do badania metod uczenia ze wzmocnieniem, w których agent uczy się strategii działania wyłącznie na podstawie nagród otrzymywanych za swoje decyzje [1].

Przełomem w tej dziedzinie było opublikowanie w 2015 roku algorytmu Deep Q-Network (DQN). Agent DQN, otrzymując na wejściu jedynie obraz ekranu i wynik gry, nauczył się grać w 49 gier z konsoli Atari 2600 i w wielu z nich osiągnął poziom porównywalny z profesjonalnym graczem [2]. Kluczowym pomysłem było połączenie klasycznego algorytmu Q-learning [3] z głęboką siecią neuronową, która przybliża wartości akcji w stanach zbyt licznych, aby można je było zapisać w tablicy. Kolejne prace pokazały, że uczenie ze wzmocnieniem pozwala tworzyć agentów pokonujących mistrzów świata w wywodzącej się z Chin grze Go [4] oraz osiągających poziom najlepszych graczy w złożonej grze strategicznej czasu rzeczywistego StarCraft II [5].

Osiągnięcia te wymagały jednak ogromnych zasobów obliczeniowych, niedostępnych poza dużymi laboratoriami badawczymi. Aby samodzielnie zbadać działanie algorytmu DQN, potrzebne jest środowisko, które z jednej strony stawia agentowi rzeczywiste wyzwania, a z drugiej pozwala przeprowadzić trening na komputerze osobistym. Takim środowiskiem jest gra Snake. Jej reguły są proste: wąż porusza się po planszy, zjada pożywienie i rośnie, a gra kończy się w chwili zderzenia ze ścianą lub z własnym ciałem. Mimo to gra wymaga od agenta planowania kilku ruchów naprzód, unikania zagrożeń, które zmieniają się wraz z długością węża, oraz łączenia celu krótkoterminowego (zdobycie pożywienia) z długoterminowym (przetrwanie).

Dodatkową motywacją do podjęcia tematu jest możliwość praktycznego sprawdzenia rozwiązań zaproponowanych w literaturze jako ulepszenia podstawowego algorytmu DQN, takich jak Double DQN [6], oraz porównania własnej implementacji z dostępnymi rozwiązaniami bibliotecznymi.

Grę Snake można rozwiązywać również bez uczenia, metodami klasycznego przeszukiwania. Algorytm A* [**?**] wyznacza najkrótszą ścieżkę od głowy węża do pożywienia, korzystając z pełnej wiedzy o planszy i heurystyki odległości. Zestawienie go z agentem DQN, który nie zna reguł gry i uczy się wyłącznie na podstawie nagród, pozwala ocenić, co zyskuje się, a co traci, zastępując zaprogramowaną strategię strategią wyuczoną.

1.2. Struktura pracy

Praca składa się z sześciu rozdziałów. W rozdziale 2 przedstawiono podstawy teoretyczne uczenia ze wzmocnieniem oraz omówiono wykorzystanie tych metod w grach komputerowych. Rozdział 3 zawiera przegląd algorytmów uczenia ze wzmocnieniem, klasycznych metod przeszukiwania i bibliotek programistycznych, zakończony uzasadnieniem wyboru algorytmu DQN. W rozdziale 4 opisano projekt systemu: środowisko gry, reprezentację stanu, funkcję nagrody i architekturę aplikacji. Rozdział 5 poświęcono implementacji środowiska, sieci neuronowej, agenta i procesu uczenia.

Ostatni rozdział zawiera podsumowanie pracy i wnioski.

1.3. Cel i zakres pracy

Celem pracy było przedstawienie wykorzystania algorytmów uczenia ze wzmocnieniem w grach komputerowych, dokonanie przeglądu tych algorytmów oraz dostępnych rozwiązań bibliotecznych, a także implementacja algorytmu DQN do budowania strategii agenta gry Snake w języku Python, oraz porównanie go z agentem wykorzystującym algorytm przeszukiwania A*. Zakres pracy obejmował opracowanie środowiska gry w wersji graficznej i uproszczonej, przyspieszonej na potrzeby treningu, zaprojektowanie reprezentacji stanu i funkcji nagrody, implementację agenta DQN w wariancie Double DQN z wykorzystaniem biblioteki PyTorch oraz przygotowanie procesu uczenia z możliwością wykorzystania procesora graficznego.

2. Uczenie ze wzmocnieniem w grach komputerowych

2.1. Podstawy uczenia ze wzmocnieniem

Uczenie ze wzmocnieniem jest dziedziną uczenia maszynowego, w której agent uczy się podejmować sekwencje decyzji na podstawie interakcji ze środowiskiem [1]. W odróżnieniu od uczenia nadzorowanego agent nie otrzymuje przykładów poprawnych odpowiedzi. Jedyną informacją zwrotną jest skalarna nagroda, którą środowisko przekazuje po każdej wykonanej akcji. Celem agenta jest takie dobieranie akcji, aby suma nagród uzyskanych w długim horyzoncie czasowym była jak największa.

Interakcja przebiega w dyskretnych krokach czasowych $t = 0, 1, 2, \ldots$ W każdym kroku agent obserwuje stan środowiska s_t, wybiera akcję a_t, a środowisko odpowiada nagrodą r_{t+1} oraz nowym stanem s_{t+1}. Schemat ten powtarza się aż do osiągnięcia stanu terminalnego, który kończy epizod.

2.1.1. Proces decyzyjny Markowa

Formalnym modelem środowiska w uczeniu ze wzmocnieniem jest proces decyzyjny Markowa (ang. *Markov decision process*, MDP). Zgodnie ze wzorem (2.1) opisuje go krotka pięciu elementów [1]:

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma), \tag{2.1}$$

gdzie: $\mathcal{S}$ – zbiór stanów, $\mathcal{A}$ – zbiór akcji, $P(s' \mid s, a)$ – prawdopodobieństwo przejścia do stanu s' po wykonaniu akcji a w stanie s, $R(s, a)$ – oczekiwana nagroda za wykonanie akcji a w stanie s, $\gamma \in [0, 1)$ – współczynnik dyskontowania.

Nazwa procesu pochodzi od własności Markowa: następny stan zależy wyłącznie od stanu bieżącego i wybranej akcji, a nie od całej wcześniejszej historii. W praktyce oznacza to, że stan musi zawierać wszystkie informacje potrzebne do podjęcia decyzji.

W grze Snake rozważanej w niniejszej pracy elementy krotki (2.1) mają następującą interpretację:

- stan s_t to wektor 11 cech binarnych opisujących zagrożenia wokół głowy węża, kierunek ruchu i położenie jedzenia (szczegóły w rozdziale 4),
- zbiór akcji $\mathcal{A} = \{\text{prosto}, \text{w prawo}, \text{w lewo}\}$ zawiera skręty względem bieżącego kierunku ruchu,
- nagroda wynosi $+10$ za zjedzenie pożywienia, -10 za kolizję oraz $\pm 0{,}1$ za zbliżenie się do pożywienia lub oddalenie od niego,

– epizod kończy się kolizją ze ścianą lub z ciałem węża.

2.1.2. Zwrot i współczynnik dyskontowania

Miarą jakości zachowania agenta od chwili t jest zwrot (ang. *return*), czyli zdyskontowana suma przyszłych nagród, określona wzorem (2.2):

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \ldots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}. \tag{2.2}$$

Występujący we wzorze (2.2) współczynnik γ określa, jak bardzo agent ceni nagrody odległe w czasie. Przykładowo nagroda +10 otrzymana za 50 kroków jest dziś warta $10 \cdot 0{,}99^{50} \approx 6{,}05$ przy $\gamma = 0{,}99$, ale zaledwie $10 \cdot 0{,}9^{50} \approx 0{,}05$ przy $\gamma = 0{,}9$. Wartość bliska jedności sprzyja więc planowaniu długoterminowemu, co w grze Snake ma znaczenie, ponieważ droga do pożywienia często wymaga wielu ruchów.

2.1.3. Strategia i funkcje wartości

Sposób postępowania agenta opisuje strategia (polityka) π, która każdemu stanowi przyporządkowuje akcję $a = \pi(s)$ lub rozkład prawdopodobieństwa akcji $\pi(a \mid s)$. Do oceny strategii służą funkcje wartości [1]. Funkcja wartości stanu, dana wzorem (2.3), określa oczekiwany zwrot przy rozpoczęciu w stanie s i dalszym postępowaniu zgodnie ze strategią π:

$$V^{\pi}(s) = \mathbb{E}_{\pi}\left[G_t \mid s_t = s\right]. \tag{2.3}$$

W algorytmach opisanych w dalszej części pracy kluczowa jest funkcja wartości akcji, zwana funkcją Q. Wzór (2.4) określa ją jako oczekiwany zwrot po wykonaniu akcji a w stanie s i dalszym postępowaniu zgodnie z π:

$$Q^{\pi}(s,a) = \mathbb{E}_{\pi}\left[G_t \mid s_t = s, a_t = a\right]. \tag{2.4}$$

Funkcja (2.4) ma bezpośrednie znaczenie praktyczne: znając jej wartości dla wszystkich akcji w danym stanie, agent może wybrać akcję najlepszą bez znajomości modelu środowiska.

2.1.4. Równania Bellmana

Ze wzoru (2.2) wynika rekurencyjna zależność $G_t = r_{t+1} + \gamma G_{t+1}$. Po podstawieniu jej do definicji (2.4) otrzymuje się równanie Bellmana (2.5) [1]:

$$Q^{\pi}(s,a) = \mathbb{E}_{\pi}\left[r_{t+1} + \gamma\, Q^{\pi}(s_{t+1}, a_{t+1}) \mid s_t = s, a_t = a\right]. \tag{2.5}$$

Zgodnie z równaniem (2.5) wartość akcji jest sumą natychmiastowej nagrody i zdyskontowanej wartości sytuacji, do której ta akcja prowadzi.

Strategia optymalna π^* to taka, która w każdym stanie zapewnia największy oczekiwany zwrot. Odpowiadająca jej optymalna funkcja wartości akcji Q^* spełnia równanie optymalności Bellmana (2.6):

$$Q^*(s,a) = R(s,a) + \gamma \sum_{s' \in \mathcal{S}} P(s' \mid s,a) \max_{a' \in \mathcal{A}} Q^*(s',a'). \tag{2.6}$$

Gdy Q^* jest znana, strategię optymalną wyznacza się według wzoru (2.7), wybierając w każdym stanie akcję o największej wartości:

$$\pi^*(s) = \arg\max_{a \in \mathcal{A}} Q^*(s,a). \tag{2.7}$$

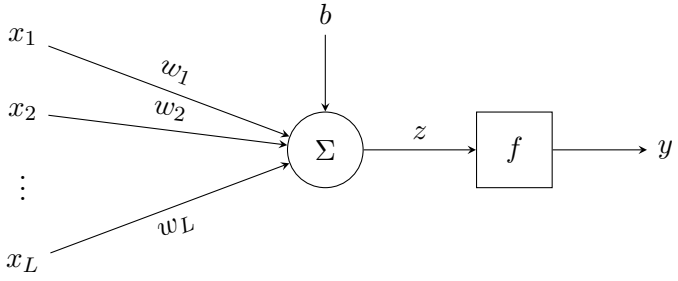
Bezpośrednie rozwiązanie równania (2.6) wymaga znajomości funkcji przejścia P i nagród R, które w grach zwykle nie są dane w jawnej postaci. Dlatego w praktyce stosuje się metody uczące się wartości Q^* na podstawie doświadczeń zbieranych podczas interakcji ze środowiskiem. Najważniejsze z nich przedstawiono w rozdziale 3.

2.2. Sieci neuronowe

Metody uczenia ze wzmocnieniem wykorzystywane w praktyce, w tym algorytm DQN [2], przybliżają funkcję Q za pomocą sztucznych sieci neuronowych. W niniejszym podrozdziale przedstawiono podstawowe pojęcia z tego zakresu: model neuronu, sieć wielowarstwową, funkcje aktywacji oraz sposób uczenia sieci.

2.2.1. Model neuronu

Podstawowym elementem sieci jest sztuczny neuron, którego schemat przedstawiono na rysunku 2.1. Neuron otrzymuje L sygnałów wejściowych $x_1, \ldots, x_L$, z których każdy jest mnożony przez odpowiadającą mu wagę w_j. Ważone sygnały są sumowane wraz z przesunięciem b (ang. *bias*).



Rys. 2.1. Model sztucznego neuronu. Źródło: opracowanie własne na podstawie [7]

Wynik sumowania, nazywany łącznym pobudzeniem neuronu, oznaczono symbolem z i opisano wzorem (2.8) [7]:

$$z = \sum_{j=1}^{L} w_j x_j + b. \tag{2.8}$$

Sygnał wyjściowy neuronu powstaje przez podanie pobudzenia (2.8) na funkcję aktywacji f, zgodnie ze wzorem (2.9):

$$y = f(z). \tag{2.9}$$

2.3. Gry jako środowisko badawcze

2.4. Przykłady zastosowań

3. Przegląd algorytmów i bibliotek

3.1. Algorytmy tablicowe

3.2. Głębokie algorytmy oparte na wartości

3.3. Metody gradientu strategii

3.4. Biblioteki uczenia ze wzmocnieniem

3.5. Uzasadnienie wyboru rozwiązania

4. Projekt systemu

4.1. Środowisko gry Snake

4.2. Reprezentacja stanu

4.3. Funkcja nagrody

4.4. Architektura systemu

5. Implementacja

5.1. Implementacja środowiska gry

5.2. Sieć neuronowa

5.3. Agent DQN

5.4. Proces uczenia

5.5. Interfejs graficzny

6. Podsumowanie i wnioski

Autor za własny wkład pracy uważa:

Literatura

[1] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, 2 edition, 2018.

[2] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.

[3] Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3–4):279–292, 1992.

[4] David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016.

[5] Oriol Vinyals, Igor Babuschkin, Wojciech M. Czarnecki, Michaël Mathieu, Andrew Dudzik, Junyoung Chung, David H. Choi, Richard Powell, Timo Ewalds, Petko Georgiev, et al. Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature*, 575(7782):350–354, 2019.

[6] Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double q-learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 30, pages 2094–2100, 2016.

[7] Roman Zajdel. Sztuczna inteligencja. laboratorium, ćwiczenie 4: Model neuronu. Politechnika Rzeszowska, `https://materialy.prz-rzeszow.pl/pracownik/pliki/34/%C4%86WICZENIE%2061.pdf`. Dostęp: 9.11.2026.

POLITECHNIKA RZESZOWSKA im. I. Łukasiewicza Rzeszów, 2026
Wydział Elektrotechniki i Informatyki

STRESZCZENIE PRACY DYPLOMOWEJ INŻYNIERSKIEJ

IMPLEMENTACJA ALGORYTMU DQN DO BUDOWANIA STRATEGII AGENTA GRY KOMPUTEROWEJ

Autor: Kacper Hyliński, nr albumu: 177082
Opiekun: dr hab. inż. Roman Zajdel prof. PRz
Słowa kluczowe: uczenie ze wzmocnieniem, Deep Q-Network, Double DQN, gra Snake, PyTorch

Praca dotyczy zastosowania uczenia ze wzmocnieniem w grach komputerowych. Przedstawiono podstawy teoretyczne uczenia ze wzmocnieniem, przykłady jego wykorzystania w grach oraz przegląd algorytmów i bibliotek programistycznych. W części praktycznej zaimplementowano w języku Python agenta grającego w grę Snake z wykorzystaniem algorytmu Deep Q-Network w wariancie Double DQN. Opracowano środowisko gry w wersji graficznej oraz uproszczonej, przyspieszającej trening, a także reprezentację stanu w postaci 11 cech binarnych i funkcję nagrody z elementami kształtowania. Sieć neuronową zrealizowano w bibliotece PyTorch z możliwością prowadzenia treningu na procesorze graficznym.

RZESZOW UNIVERSITY OF TECHNOLOGY Rzeszow, 2026
Faculty of Electrical and Computer Engineering

BSC THESIS ABSTRACT

IMPLEMENTATION OF THE DQN ALGORITHM FOR BUILDING A COMPUTER GAME AGENT'S STRATEGY

Author: Kacper Hyliński, Student's ID: 177082
Supervisor: Roman Zajdel, DSc, Eng., Associate Professor
Key words: reinforcement learning, Deep Q-Network, Double DQN, Snake game, PyTorch

The thesis concerns the application of reinforcement learning in computer games. It presents the theoretical foundations of reinforcement learning, examples of its use in games, and a review of algorithms and software libraries. In the practical part, an agent playing the game Snake was implemented in Python using the Deep Q-Network algorithm in its Double DQN variant. A game environment was developed in a graphical version and a simplified version that accelerates training, together with a state representation of 11 binary features and a shaped reward function. The neural network was implemented in PyTorch with support for GPU training.